

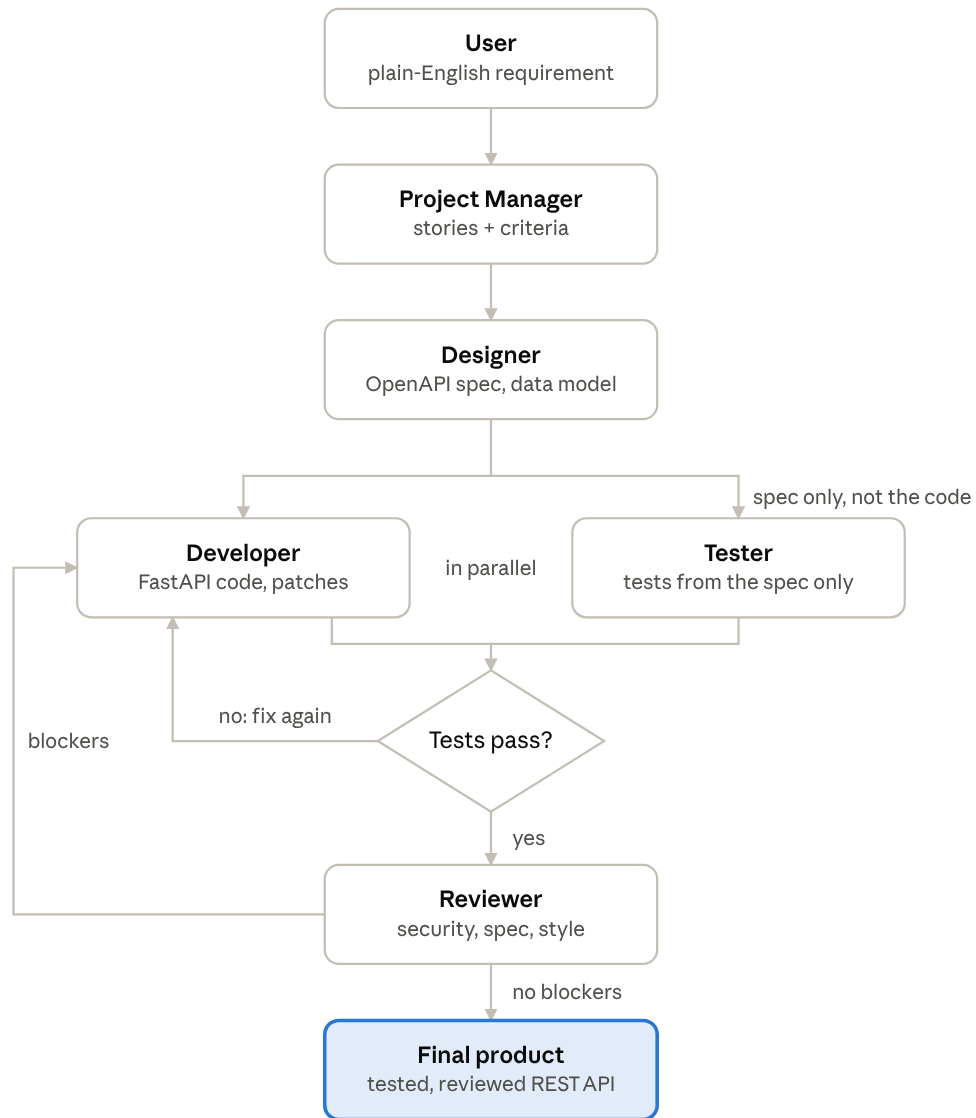
Multi-Agent Software Team: Project Plan

Oct 8, 2026 · @Aashutosh

Build five cooperating LLM agents that turn a plain-English requirement into a tested, reviewed REST API, in about six weeks, using eight published papers as the design basis.

Architecture

Work flows down the page, and both failed tests and reviewer blockers send the build back to the Developer.

Work flows down; failed tests and blockers loop back to the Developer

agent hand-offs · 5 agents, 1 test gate, 2 loops back

The Tester works from the spec in parallel with the Developer, so neither can copy the other's mistakes.

Agent roles

Each agent has one job, a fixed input and output format, and a measurable "done" condition, so the team can be tested agent by agent.

Agent	Takes in	Produces	Tools	Done when
Project Manager	User requirement, status reports from all agents	User stories, acceptance criteria, task board, final summary	Task board (JSON), message router	Every acceptance criterion maps to a passing test
Designer	User stories	OpenAPI spec, data model, folder structure	OpenAPI validator	Spec validates and covers every story
Developer	OpenAPI spec, assigned task, failure reports	FastAPI code, database models	File writer, Docker shell, linter	Code starts, lints clean, and the Tester's suite passes
Tester	OpenAPI spec and stories (not the code)	pytest suite, coverage report, bug reports	pytest, coverage, Docker shell	Each endpoint has success, validation and error tests
Reviewer	Code, tests, test results	Review comments ranked blocker / major / minor	Static analysis (ruff, bandit), diff reader	No blockers left and spec conformance confirmed

The Tester works from the spec, not from the Developer's code, so the tests cannot simply confirm the code's own mistakes. The Designer here means API and architecture designer; a front-end mockup is an optional extension.

Suggested stack: Python, LangGraph for the agent graph (AutoGen is the alternative), FastAPI + SQLite + pytest as the generated target, Docker as the sandbox, and any LLM API.

Research papers

Eight papers cover role-based design, test-driven loops, tool interfaces and evaluation; read the first three before writing code. Titles and authors were checked against their arXiv pages.

#	Paper	First author, year	What to borrow
1	ChatDev: Communicative Agents for Software Development	Qian, 2023	A fixed chain of phases (design, coding, testing) and "communicative dehallucination": an agent asks clarifying questions before acting
2	MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework	Hong, 2023	Standard operating procedures written into prompts, and structured documents (requirements, design) passed between roles instead of free chat
3	AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation	Huang, 2023	A tester that writes tests independently of the code, and an executor that feeds failures back to the programmer
4	Self-collaboration Code Generation via ChatGPT	Dong, 2023	Analyst, coder and tester roles set by role instructions; use it as the simple baseline your team must beat
5	AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation	Wu, 2023	Conversable agents and routing patterns (group chat, hand-offs); the alternative to LangGraph
6	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	Yang, 2024	A small, well-designed set of file view, edit and test commands with short feedback, instead of raw shell access
7	Reflexion: Language Agents with Verbal Reinforcement Learning	Shinn, 2023	After each failed test run, store a short written reflection and give it to the next attempt
8	SWE-bench: Can Language Models Resolve Real-World GitHub Issues?	Jimenez, 2023	Evaluating on real GitHub issues, as an optional stretch goal beyond the task-manager API

Execution plan

Six phases over about six weeks (a suggested pace for one person), each ending in a gate you can check before moving on.

Phase	Weeks	Deliverable	Exit gate
0. Foundations	1	Repo, Docker sandbox, LLM wrapper, logging, single-agent baseline	Baseline generates an API for the task-manager prompt and its test result is recorded
1. Contracts and orchestration	2	Message schemas, agent graph with stub agents	A stub run passes through every node and logs each message
2. Core agents	2 to 3	Project Manager, Designer, Developer	A requirement becomes a valid OpenAPI spec and code that starts in the sandbox
3. Tester and fix loop	4	Tester, sandbox runner, failure reports, reflection memory	The loop ends with all tests green or at the retry cap, never hangs
4. Reviewer and quality gates	5	Reviewer, static analysis, escalation rules	Reviewer flags every bug you deliberately planted in a sample project
5. Evaluation and portfolio	6	Benchmark run, comparison with baseline, demo, README	Results table and demo recording published in the repo

Workflow of each implementation step

Do the steps in order; each phase's last step is its gate from the table above.

Phase 0: Foundations

1. Create the repo with folders for agents, graph, schemas, sandbox, evals and outputs.
2. Build a Docker image with Python, pytest, ruff and bandit; run generated code there with no network and a time limit.

3. Write one LLM wrapper that logs the prompt, response, token count and latency of every call.
4. Write the task-manager requirement and its acceptance criteria as your first test input.
5. Build the single-agent baseline (one prompt, one code output), run it in the sandbox and save the result.

Phase 1: Contracts and orchestration

1. Define typed messages (Pydantic models): Requirement, UserStories, ApiSpec, CodeBundle, TestSuite, TestReport, ReviewReport.
2. Create one graph node per agent that returns a fixed dummy output.
3. Wire the edges: Project Manager, then Designer, then Developer and Tester in parallel, then test run, then Reviewer, then back to the Project Manager.
4. Add a conditional edge from a failed test run back to the Developer, and a shared retry counter in the graph state.
5. Run the stub graph end to end and read the log to confirm every message arrives in the right shape.

Phase 2: Core agents

1. Project Manager: prompt it to turn the requirement into stories with testable acceptance criteria, and to ask the user a clarifying question when something is ambiguous (from ChatDev).
2. Designer: generate the OpenAPI spec and validate it; on an error, send the validator message back and regenerate.
3. Developer: generate the files from the spec using a small command set (read file, write file, run) in the style of SWE-agent.
4. Start the app in the sandbox and check that a health endpoint responds.
5. Save every agent's output as a document in the outputs folder, so each hand-off can be inspected (from MetaGPT).

Phase 3: Tester and fix loop

1. Tester: generate pytest files from the spec and stories only, without reading the code.
2. Runner: execute the suite in the sandbox and parse the result into a TestReport with test name, error and a short trace.

3. On failure, have the Developer write a two or three sentence reflection on the cause and keep it in memory for the next attempt (from Reflexion).
4. Send the report and reflection to the Developer and ask for a patch limited to the failing parts.
5. Add a triage step: the Project Manager decides whether a failure is a code bug (to the Developer) or a test bug (to the Tester).
6. Repeat until all tests pass or the retry cap is reached; at the cap, return an "unresolved" report instead of looping.

Phase 4: Reviewer and quality gates

1. Give the Reviewer the code, spec, tests and the ruff and bandit output.
2. Ask it to check that every spec endpoint exists and every acceptance criterion has a test.
3. Have it rank comments as blocker, major or minor; blockers go back to the Developer, the rest go into the final report.
4. Plant known bugs in a sample project (missing input validation, missing auth check, unsafe SQL) and confirm the Reviewer finds each one.
5. The Project Manager accepts the build only when no blockers remain.

Phase 5: Evaluation and portfolio

1. Write a set of requirements of rising difficulty, starting with the task-manager API.
2. Run the baseline and the full team on each; record tests passed, endpoints covered, attempts used, tokens and wall time.
3. Run ablations: remove the Tester, then the Reviewer, to show what each agent adds.
4. Add a simple CLI or web page that shows the agents' conversation as it happens.
5. Write the README with the architecture diagram, results table and known limits, and record a short demo.

Evaluation and portfolio package

The portfolio claim is "the team beats a single agent", so measure both on the same requirements.

Metric	How to measure
Test pass rate	Share of the Tester's tests that pass at the end of the run
Spec coverage	Endpoints in the OpenAPI spec that exist in the running app
Attempts to green	Number of Developer fix rounds before all tests pass
Reviewer recall	Planted bugs the Reviewer flagged out of the planted total
Cost	Tokens and wall-clock time per run, from the LLM wrapper's log

Suggested repository layout:

```

agent-dev-team/
  agents/      # one file per agent: prompt + tools
  graph/       # orchestration, routing, retry logic
  schemas/     # typed messages between agents
  sandbox/     # Dockerfile and runner
  evals/       # requirements, baseline, ablations, results
  outputs/     # generated specs, code, tests, reviews per run
  README.md    # architecture, results table, limits, demo link

```

Risks

Risk	Fallback
Agents loop without converging	Retry cap per phase and an "unresolved" report to the Project Manager
The Tester writes a wrong test and the Developer "fixes" correct code	Triage step; Tester works from the spec only; spot-check a sample of tests by hand
Generated code is unsafe to run	Docker sandbox with no network, and time and memory limits

Risk	Fallback
Cost grows with every extra round	Log tokens per call; try a smaller model for simpler roles and compare results
One lucky run looks better than it is	Run each requirement several times and report the spread, not the best run
Scope creep into front-end or deployment	Keep the target a REST API; treat the Designer's UI mockups as a later extension